Block 1 — Week 3

SQL for Health Data

Essentials of Biomedical Informatics — Self-Paced Module (Asynchronous)

Carlos E. Figueroa Castro, MD. MS
Associate Professor, Infectious Diseases
Medical College of Wisconsin
Milwaukee, WI

Overview

Last week you explored the OMOP structure. This week you build a real cohort definition with Structured Query Language (SQL) — the single most common task in observational health data science — and connect that SQL to both R and Python. Plan for roughly 3–4 hours.

Companion Files

- synthetic_omop.duckdb — same database as Week 2
- Week3_Cohort_Query.sql — worked SQL walkthrough + exercise (common table expressions, window functions)
- Week3_Cohort_R.qmd — running the same logic from R via DBI
- Week3_Cohort_Python.ipynb — running the same logic from Python via duckdb + pandas

Complete the SQL file, then ONE of the R or Python files (or both, if you want the practice).

Learning Objectives

- Use common table expressions (WITH ... AS) to break a complex query into readable, named steps
- Use a window function (ROW_NUMBER) to find each patient's first occurrence of an event
- Build a clinical cohort definition combining conditions, drug exposures, and a time window
- Connect to a database from R (DBI) and Python (duckdb) and load query results into a data frame
- Explain, at a high level, why filtering happens in the database rather than after loading all data into memory

Core Concepts

A Common Table Expression (CTE), written as WITH name AS (...), lets you name an intermediate result and reference it later in the same query — essentially a labeled scratchpad. Instead of nesting subqueries inside subqueries, you write a sequence of clearly named steps, which is both easier to read and easier to debug one piece at a time.

A window function computes a value across a set of rows related to the current row, without collapsing them the way GROUP BY does. ROW_NUMBER() OVER (PARTITION BY person_id ORDER BY some_date) numbers each patient's events in order — which is exactly what you need to find someone's FIRST diagnosis, FIRST prescription, or FIRST lab result, a extremely common need in cohort research (e.g., “new-user” study designs, which only include patients starting a treatment for the first time).

Putting these together, a cohort definition is just a precise, reproducible rule for “who is in my study population” — for example, “patients diagnosed with Type 2 Diabetes who started metformin within 90 days.” Every finding in observational research depends entirely on how the cohort was defined, which is why writing (and documenting) this logic carefully is a core skill, not a technicality.

Finally, a practical point that matters more as datasets grow: push filtering and joining into the database with SQL, and only pull back the rows and columns you actually need into R or Python. On a real OMOP database with hundreds of millions of rows, loading full tables into memory before filtering is often simply not possible.

Setup

Builds on Week 2's setup. One addition for the R track:

- R: `install.packages(c("DBI", "duckdb", "dplyr"))`

- Python: duckdb and pandas (already installed in Week 1/2)

This Week's Deliverable

Work through Week3_Cohort_Query.sql Part 1, then complete Part 2 (Your Turn) there. Then complete the matching "Your Turn" section in EITHER Week3_Cohort_R.qmd OR Week3_Cohort_Python.ipynb:

1. Adapt the cohort query for Hypertension → Lisinopril within 180 days.
2. Report the mean age at diagnosis by sex.
3. Export the cohort to a CSV file (you'll reuse this in Week 4).

Submit the completed .sql file plus your chosen .qmd/.html or .ipynb file via the Week 3 dropbox in Brightspace.

Self-Check Quiz

Use this to check your own understanding before submitting. Answer the quiz in Brightspace.

1. What does a CTE (WITH ... AS) let you do?

- a) Permanently save a table to disk
- b) Name an intermediate result so you can reference it later in the same query
- c) Automatically index a table
- d) Connect to a remote server

2. What does ROW_NUMBER() OVER (PARTITION BY person_id ORDER BY date) compute?

- a) The total number of patients in the table
- b) A running count of ALL rows, ignoring which patient they belong to
- c) A rank for each patient's own events, restarting at 1 for every new patient
- d) The average date per patient

3. Why is finding a patient's FIRST prescription of a drug often important in cohort research?

- a) It's required by SQL syntax
- b) "New-user" designs, which only include patients starting a treatment for the first time, reduce certain biases in observational research
- c) It makes the query run faster
- d) It is not actually important

4. What is a "cohort definition" in observational research?

- a) A visual chart of patient counts
- b) A precise, reproducible rule specifying which patients are included in a study population
- c) A list of every table in a database
- d) A type of machine learning model

5. Why prefer filtering data in SQL over pulling entire tables into R/Python first?

- a) SQL syntax is easier to read

- b) On large databases, loading everything into memory before filtering is often not feasible; the database engine filters efficiently on disk
- c) R and Python cannot read large files
- d) It is not actually preferable

Suggested Readings

- SQLZoo interactive tutorials: https://sqlzoo.net/wiki/SQL_Tutorial
- Observational Health Data Sciences and Informatics. (2021, January 11). The Book of OHDSI.
- https://ohdsi.github.io/TheBookOfOhdsi/SqlAndR.html (chapter 9)

Suggested Grading Rubric

A pass grade required 60/100 points

Criterion	Weight	What to look for
SQL walkthrough completion	15%	All Part 1 queries run without errors
SQL Task 1/2: cohort + stratification	30%	Correct CTE/window-function logic; correct grouping
SQL Task 3: written explanation	15%	Correctly explains ROW_NUMBER vs MIN()
R or Python: cohort reproduced + exported	40%	Query adapted correctly; valid CSV export